Zero-Downtime Infrastructure Updates

Terraform Infrastructure as Code and Operational Automation

Project explained in simple terms: summary, components, workflow and architecture

> **The project in one line**
>
> Shipped 300+ in-service updates with zero customer-facing downtime, as measured by change-management records, by building Terraform Infrastructure as Code and operational automation for safe patching, canary rollouts, automatic failover, and rollbacks.

Headline result	Value
Updates delivered while the service stayed live	300+
Customer-facing downtime	Zero
How it was measured	Change-management records (one ticket per change, with evidence)
How it was achieved	Terraform IaC + automation for patching, canary, failover and rollback

Contents

Section	What you will find
1. Overall Summary in Simple Terms	The whole project explained in plain words
2. Key Terms Explained Simply	Glossary of every technical word used
3. Architecture Diagram	The full picture of the system (landscape page)
4. Architecture Diagram Explained in Detail	Every box, colour, arrow and label explained
5. Components Explained in Detail	All 16 components: what, analogy, details, why it matters
6. Inside the Terraform Code	Code layout and simple example snippets
7. The Four Automation Playbooks	Safe patching, canary rollouts, automatic failover, rollbacks
8. End-to-End Workflow	Workflow diagram and all 13 steps, plus failure handling
9. How Zero Downtime Was Measured	How change-management records prove the result
10. Explaining It in an Interview	A 30-second pitch and likely follow-up questions

1. Overall Summary in Simple Terms

Imagine an online service that thousands of customers use all day and night. Like a car, it needs regular servicing: security patches, software upgrades and setting changes. The old way was to switch things off, fix them and switch them back on. That means customers see errors or a 'down for maintenance' page.

This project built a system that services the car **while it is still driving**, without passengers noticing. It did this more than **300 times** with **zero customer-facing downtime**, and every single change is recorded in the company's change-management system, so the result is proven, not guessed.

1.1 The two big ideas

- **Terraform Infrastructure as Code (IaC):** instead of building servers, networks and load balancers by clicking in a cloud console, everything is written down as code. The code is the blueprint. Terraform reads the blueprint and builds or changes the real infrastructure to match it, the same way every time.
- **Operational automation:** scripts and pipelines that carry out the risky parts of a change automatically and safely, following four safety patterns.

1.2 The four safety patterns

Pattern	What it means in simple terms	Everyday analogy
Safe patching	Update servers a few at a time. Before touching a server, move its customers to other servers. Put it back only once it is healthy.	Repairing one lane of a motorway while traffic uses the other lanes.
Canary rollouts	Send only a small share of customers (for example 5%) to the new version first. Watch closely. Only then increase to 25%, 50%, 100%.	Taste-testing a new dish with a few diners before putting it on the full menu.
Automatic failover	If a server, zone or database fails, traffic moves to a healthy copy automatically, in seconds.	A backup generator that starts by itself when the power cuts out.
Rollbacks	If the new version looks worse, the system automatically switches back to the last good version within minutes.	An undo button.

1.3 Why it worked

- Nothing is done by hand in production. Everything is code, reviewed and tested first.

- There are always at least two copies of everything (servers, zones, database), so one can be updated while the other serves customers.
- Changes reach customers gradually, and are measured at every step.
- If anything looks wrong, the system undoes itself before customers notice.
- Every change has a ticket with evidence, which is how 'zero downtime' is proven.

One-paragraph version

I wrote our infrastructure as Terraform code and built automation that updates the live system in small, measured steps: build a patched image, test it in staging, send 5% of real traffic to it, watch the health metrics, then ramp up or automatically roll back. Load balancers drain traffic away before any server is touched, multi-zone servers and a replicated database fail over automatically, and every change is logged in change management. Across 300+ updates, customers never saw an outage.

2. Key Terms Explained Simply

Term	Simple meaning
Infrastructure	All the 'plumbing' a service runs on: servers, networks, load balancers, databases, DNS.
Infrastructure as Code (IaC)	Describing infrastructure in text files that a tool can read and build, instead of setting it up by hand.
Terraform	A popular IaC tool. You write what you want; Terraform works out what to create, change or delete to get there.
Module	A reusable Terraform building block, for example 'a load balancer with standard settings'.
terraform plan	A preview: 'here is exactly what I would change'. Nothing is changed yet.
terraform apply	Actually making the changes shown in the plan.
State file	Terraform's record of what it has already built, kept remotely and locked during changes.
Drift	When the real infrastructure no longer matches the code, usually because someone changed it by hand.
In-service update	An update done while the service stays live for customers.
Patching	Installing fixes, usually security fixes, for the operating system or software.
Golden image	A pre-built, pre-patched, tested server template used to create identical servers.
Load balancer	A traffic cop that spreads customer requests across healthy servers.
Connection draining	Letting a server finish its current requests before removing it, so nobody is cut off.
Health check	An automatic 'are you OK?' test run every few seconds against each server.
Availability zone	A separate data center with its own power and network, close to the others.

Term	Simple meaning
Canary	A small group running the new version, receiving a small share of traffic as an early warning.
Bake time	How long to wait and watch at each step before deciding it is safe to continue.
Failover	Automatically switching to a backup when something fails.
Rollback	Going back to the previous known-good version.
Blast radius	How much of the system a single problem could affect. Smaller is safer.
SLO	Service Level Objective: a target such as '99.9% of requests succeed'.
Change management	The formal process and system for requesting, approving and recording production changes.
CI/CD	Continuous Integration / Continuous Delivery: pipelines that test and deliver changes automatically.

3. Architecture Diagram

The diagram below shows every part of the system and how they connect. Numbers in circles match the component descriptions in the next sections. Section A is where changes are planned and approved, section B is the automation that carries them out, section C is the live production environment, and the column on the right is the safety loop that watches and protects it.

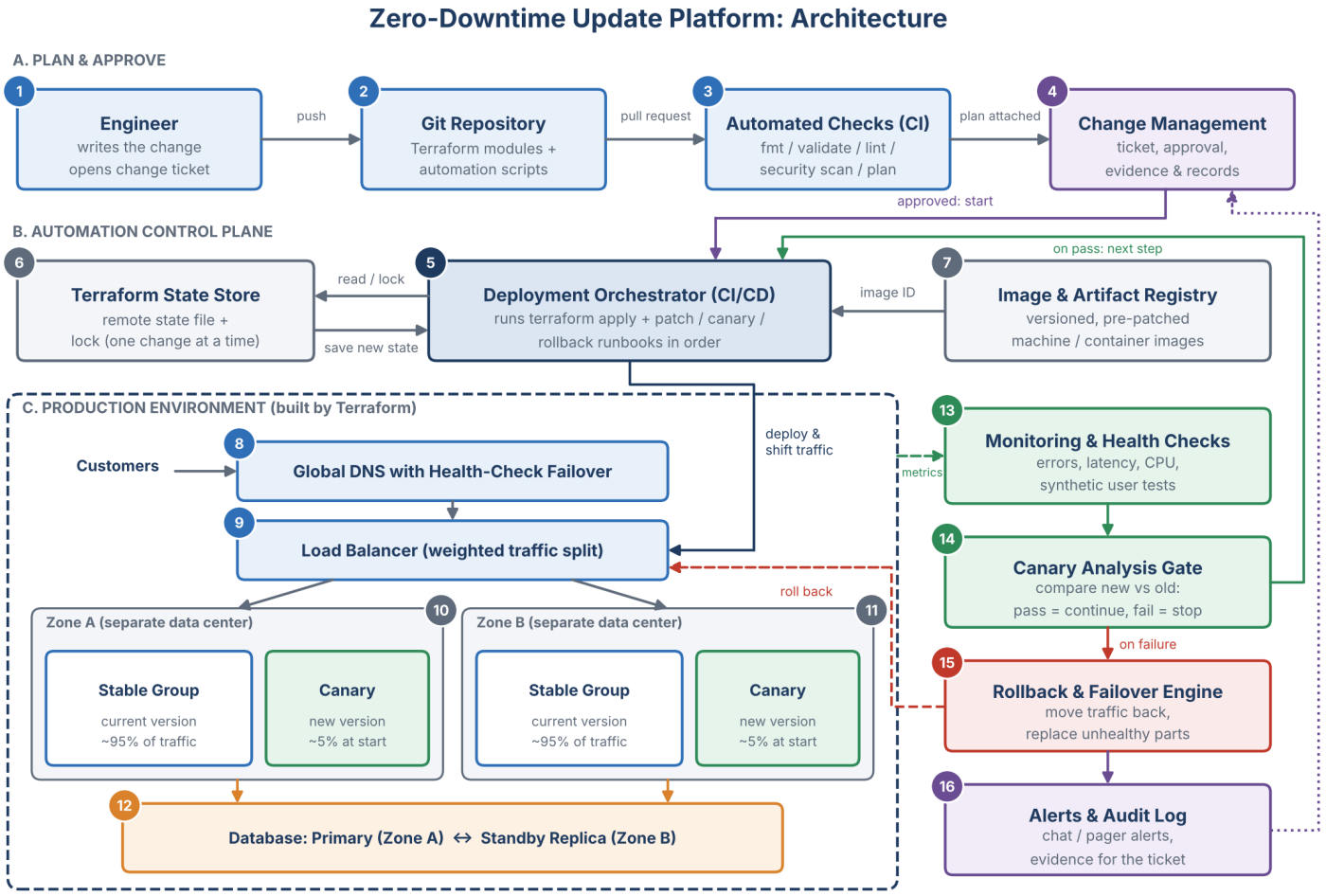


Figure 1. Zero-downtime update platform: end-to-end architecture

4. Architecture Diagram Explained in Detail

4.1 How to read the diagram

Visual element	What it means
Numbered circle (1 to 16)	The component number. Each one is explained in Section 5.
Blue boxes	Building and serving: code, checks, DNS, load balancer and stable servers.
Purple boxes and lines	Governance: change management approval and the audit trail.
Dark navy box	The Deployment Orchestrator, the 'brain' that runs every change.
Grey boxes	Supporting stores: Terraform state and the image registry; also the two zones.
Green boxes and lines	Health and progress: monitoring, canary group, canary gate, and the 'pass: continue' path.
Red box and dashed red line	Safety net: the Rollback and Failover Engine and the path traffic takes back to the old version.
Orange box	Data: the replicated database.
Solid arrow	A direct action or request (for example 'deploy' or 'send traffic').
Dashed arrow	A feedback signal or an emergency path (metrics, rollback).
Dotted purple line	Evidence being written back to the change ticket.
Dashed outline (section C)	Everything inside is live production infrastructure, all built and managed by Terraform.

4.2 Section A: Plan and Approve (top row)

This row is everything that happens **before** production is touched. It reads left to right.

- **1 Engineer → 2 Git Repository (arrow 'push')**: the engineer saves the change as code and pushes it to Git.
- **2 Git → 3 Automated Checks (arrow 'pull request')**: opening a pull request starts the CI checks automatically.
- **3 Automated Checks → 4 Change Management (arrow 'plan attached')**: the terraform plan preview and test results are attached to the change ticket so approvers see the exact effect.

- **4 Change Management → 5 Orchestrator (purple arrow 'approved: start')**: only after approval does the production pipeline get permission to run. This is the gate between planning and doing.

4.3 Section B: Automation Control Plane (middle row)

This row is the machinery that carries out the change. It never serves customers directly; it controls the parts that do.

- **5 Deployment Orchestrator** sits in the middle and runs every step in order.
- **5 ↔ 6 Terraform State Store (arrows 'read / lock' and 'save new state')**: before changing anything, the orchestrator reads the current state and locks it so no other change can run. After applying, it saves the new state and releases the lock.
- **7 Image Registry → 5 (arrow 'image ID')**: the orchestrator gets the ID of the new tested, patched image. This ID is passed to Terraform as the thing to deploy.
- **5 → 9 Load Balancer (navy arrow 'deploy and shift traffic')**: the orchestrator creates new servers and changes the load balancer weights, for example 95/5, then 75/25, and so on.
- **14 → 5 (green arrow 'on pass: next step')**: the Canary Analysis Gate tells the orchestrator it is safe to move to the next step.

4.4 Section C: Production Environment (dashed area)

This is what customers actually use. Every item inside it is defined in Terraform code.

- **Customers → 8 Global DNS**: customers type the website name; DNS answers with the address of a healthy entry point. If a whole site is unhealthy, DNS fails over to another.
- **8 DNS → 9 Load Balancer**: requests reach the load balancer, which knows which servers are healthy.
- **9 Load Balancer → 10 Zone A and 11 Zone B**: traffic is spread across two independent data centers. Inside each zone the split is by weight between the **Stable Group** (current version, about 95% at the start) and the **Canary** (new version, about 5% at the start).
- **10 and 11 → 12 Database (orange arrows)**: servers in both zones read and write data through the primary database. A standby replica in the other zone stays in sync and takes over automatically if needed.

4.5 Right column: the Safety Loop

This column is a continuous loop: **watch → decide → act → record**.

- **Production → 13 Monitoring (dashed green arrow 'metrics')**: every server, the load balancer and the database send health numbers (errors, response time, CPU) every few seconds. Synthetic tests act like robot customers.
- **13 → 14 Canary Analysis Gate**: the gate compares canary against stable. Same or better means pass; noticeably worse means fail.

- **14 → 15 Rollback and Failover Engine (red arrow 'on failure')**: a failed check triggers the engine.
- **15 → 9 Load Balancer (dashed red arrow 'roll back')**: the engine sets canary traffic back to 0%, so all customers go back to the stable version within seconds, then removes the bad servers.
- **15 → 16 Alerts and Audit Log**: every rollback or failover alerts the team and is logged.
- **16 → 4 Change Management (dotted purple line)**: all evidence goes back into the change ticket, closing the loop. This is how the 300+ zero-downtime results are measured.

4.6 The connections at a glance

From	To	Label	What travels along it
1 Engineer	2 Git	push	Code change (Terraform / scripts)
2 Git	3 CI checks	pull request	Trigger to run format, validate, lint, security scan, plan
3 CI checks	4 Change Mgmt	plan attached	Plan preview and test results
4 Change Mgmt	5 Orchestrator	approved: start	Permission to run in production
5 Orchestrator	6 State Store	read / lock	Current state; lock acquired
6 State Store	5 Orchestrator	save new state	Updated state; lock released
7 Registry	5 Orchestrator	image ID	Version of the tested, patched image
5 Orchestrator	9 Load Balancer	deploy and shift traffic	New servers and new traffic weights
Customers	8 DNS	(unlabelled)	Customer requests
8 DNS	9 Load Balancer	(unlabelled)	Requests sent to a healthy entry point
9 Load Balancer	10 / 11 Zones	(unlabelled)	Requests split by weight across stable and canary
10 / 11 Zones	12 Database	(unlabelled)	Data reads and writes
Production	13 Monitoring	metrics	Errors, latency, CPU, synthetic test results
13 Monitoring	14 Canary Gate	(unlabelled)	Side-by-side canary vs stable numbers
14 Canary Gate	5 Orchestrator	on pass: next step	Go-ahead to ramp up
14 Canary Gate	15 Rollback Engine	on failure	Stop signal

From	To	Label	What travels along it
15 Rollback Engine	9 Load Balancer	roll back	Traffic weight back to old version
15 Rollback Engine	16 Alerts / Audit	(unlabelled)	Event details
16 Alerts / Audit	4 Change Mgmt	(dotted)	Evidence attached to ticket

5. Components Explained in Detail

Each component below matches the number on the architecture diagram. For each one you will find: what it is, an everyday analogy, what it does, the fine details, and why it matters for zero downtime.

5.1 Planning and approval components

1. Engineer (the person making the change)

What it is: The operations or cloud engineer who needs to change something in production: apply a security patch, upgrade software, change a setting or add capacity.

Simple analogy: The pilot who files a flight plan before taking off.

What it does:

- Opens a change ticket that says what will change, why, when and how it will be undone if needed.
- Writes the change as code (Terraform files or automation scripts) rather than clicking around in a console.
- Pushes the code to Git and opens a pull request so others can review it.

Fine details:

- The engineer never logs into production servers to change them by hand. Every change goes through code and the pipeline.
- Because the change is code, the same steps run the same way every time, whoever runs them.

Why it matters for zero downtime: Manual changes are the most common cause of outages. Making the engineer describe the change as code removes typing mistakes and forgotten steps.

2. Git Repository (single source of truth)

What it is: A version-controlled store (for example GitHub, GitLab or Bitbucket) holding all Terraform code and automation scripts.

Simple analogy: A shared notebook where every page edit is dated, signed and can be undone.

What it does:

- Holds Terraform modules: reusable building blocks such as 'network', 'load balancer', 'server group', 'database', 'DNS', 'monitoring'.
- Holds environment folders (dev, staging, production), each using the same modules with different settings.
- Holds the runbook scripts for patching, canary, failover and rollback.
- Keeps full history: who changed what, when and why.

Fine details:

- Branch protection: nobody can push straight to the main branch; a pull request plus at least one reviewer is required.

- Every merged commit has an ID. That ID is written into the change ticket so the exact code that ran can always be found.
- Rolling back the infrastructure can be as simple as re-applying the previous commit.

Why it matters for zero downtime: If something goes wrong, the last known-good version is always one step away, and the history explains exactly what changed.

3. Automated Checks / Continuous Integration (CI)

What it is: A pipeline that runs automatically on every pull request, before a human even looks at it.

Simple analogy: A spell-checker and safety inspector that read the change before it is allowed near production.

What it does:

- **terraform fmt**: checks the code is neatly formatted.
- **terraform validate**: checks the code is valid Terraform.
- **Linting** (for example tflint): catches bad practice such as wrong instance types or missing tags.
- **Security / policy scan** (for example Checkov, tfsec or OPA policy rules): blocks risky settings such as open firewalls, unencrypted disks, or deleting a database.
- **terraform plan**: produces a preview listing exactly what will be created, changed or destroyed.

Fine details:

- The plan is posted on the pull request and attached to the change ticket so reviewers see the exact effect.
- A policy rule can fail the build if the plan wants to destroy a protected resource (for example a database), which prevents accidental deletions.

Why it matters for zero downtime: Mistakes are caught in minutes on a laptop-sized preview, not in production in front of customers.

4. Change Management System

What it is: The formal ticketing system (for example ServiceNow or Jira Service Management) that records, approves and audits every production change.

Simple analogy: Air-traffic control: no take-off without clearance, and every flight is logged.

What it does:

- Holds the change request: description, risk level, maintenance window, test evidence, rollback plan.
- Collects approvals from peers and, for higher-risk changes, a change advisory board (CAB).
- Receives evidence after the change: pipeline logs, health-check results, and whether any customer impact happened.
- Is the system of record used to prove the result: 300+ changes closed as successful with zero customer-facing downtime.

Fine details:

- Low-risk, repeatable changes (such as monthly OS patches through the proven pipeline) can be pre-approved as 'standard changes', which speeds things up safely.
- The approval itself is what unlocks the production deployment job in the pipeline.

Why it matters for zero downtime: It gives control (nothing runs without approval) and proof (every result is recorded and measurable).

5.2 Automation control plane components

5. Deployment Orchestrator (CI/CD pipeline)

What it is: The automation engine (for example Jenkins, GitHub Actions, GitLab CI, Azure DevOps or AWS CodePipeline) that runs the approved change step by step.

Simple analogy: The conductor of an orchestra: it tells each part when to play, and stops the music if something sounds wrong.

What it does:

- Runs terraform apply using the exact reviewed plan.
- Runs the runbooks in the correct order: pre-checks, canary, ramp-up, batch patching, verification.
- Waits for the Canary Analysis Gate after every step and continues only on 'pass'.
- Calls the Rollback Engine immediately on 'fail'.

Fine details:

- Runs with a dedicated, least-privilege service account, not a person's credentials.
- Only one production run at a time (concurrency lock), so two changes never collide.
- Every step's output is saved as a log and linked to the change ticket.

Why it matters for zero downtime: Machines follow the safe procedure exactly the same way every time, including at 2 a.m., and react to problems in seconds.

6. Terraform State Store and Lock

What it is: A remote, shared file (for example in an S3 bucket with a DynamoDB lock table, an Azure Storage blob or Terraform Cloud) that records what Terraform has built.

Simple analogy: The building's official floor plan, kept in one safe with a single key.

What it does:

- Stores the current 'known' state of every resource Terraform manages.
- Lets Terraform compare 'what exists' with 'what the code says' so it changes only what is different.
- Locks while a change is running so no one else can change the same infrastructure at the same time.

Fine details:

- State is encrypted and versioned, so an earlier copy can be restored if needed.
- Production and staging have separate state files, so a staging mistake cannot touch production.
- Scheduled 'drift detection' runs terraform plan regularly to spot anything changed by hand outside the code.

Why it matters for zero downtime: Prevents two people from overwriting each other and keeps Terraform's picture of the world accurate, which is essential for safe changes and rollbacks.

7. Image and Artifact Registry

What it is: A store of versioned, pre-built server images (for example AWS AMIs built with Packer) or container images, already patched and tested.

Simple analogy: A warehouse of ready-to-install, factory-tested replacement parts, each with a serial number.

What it does:

- Holds 'golden images': the operating system plus security patches plus the application, baked together.
- Gives every image a version and ID, such as app-2026-10-v14.
- Keeps older versions so a rollback can point back to the previous image instantly.

Fine details:

- Images are scanned for vulnerabilities before being approved.
- Instead of patching a live server in place, the system usually builds a new server from the new image and retires the old one (the 'immutable infrastructure' approach).
- Terraform receives the new image ID as a variable; that one value is often the whole change.

Why it matters for zero downtime: Rolling back is as simple as switching the image ID back, and every server built from the same image is identical.

5.3 Production environment components

8. Global DNS with Health-Check Failover

What it is: The internet 'phone book' (for example AWS Route 53, Azure Traffic Manager or Cloudflare) that sends customers to the right place, with built-in health checks.

Simple analogy: A GPS that reroutes you automatically when a road is closed.

What it does:

- Translates the website name into the address of a healthy load balancer.
- Constantly checks endpoint health from several locations.
- If a whole site or region fails its health checks, automatically points customers to the healthy one.

Fine details:

- Records use a short TTL (time-to-live, for example 30 to 60 seconds) so customers' devices pick up a change quickly.

- Configured entirely by Terraform, including the health checks and failover rules.

Why it matters for zero downtime: Even if an entire location breaks, customers are sent somewhere that works without anyone needing to act.

9. Load Balancer with Weighted Traffic Split

What it is: The 'traffic cop' (for example an AWS Application Load Balancer, Azure Load Balancer or NGINX) that spreads requests across healthy servers.

Simple analogy: A receptionist who sends each visitor to a free desk, and never to an empty one.

What it does:

- Sends each customer request only to servers that pass health checks.
- Splits traffic by weight, for example 95% to the stable group and 5% to the canary group.
- Uses connection draining: before a server is removed, it stops sending new requests to it and lets current ones finish.

Fine details:

- Health check example: call /health every 10 seconds; 3 failures mark a server unhealthy, 2 successes mark it healthy again.
- Changing the weights is how both canary ramp-up and rollback happen. Rollback means setting the canary weight back to 0%, which takes effect in seconds.
- Connection draining (deregistration delay of, say, 30 to 300 seconds) is what stops customers seeing dropped requests.

Why it matters for zero downtime: This is the heart of zero downtime: traffic is always moved away before anything is touched.

10. Zone A: server groups in the first data center

What it is: A group of servers in one availability zone (a physically separate data center), split into a stable group and a canary group.

Simple analogy: One of two kitchens in a restaurant. Either kitchen alone can still serve the diners.

What it does:

- **Stable group**: runs the current, trusted version and carries most of the traffic.
- **Canary group**: runs the new version and carries a small share of traffic at first.
- Uses auto scaling to keep the right number of healthy servers and replace failed ones.

Fine details:

- Capacity is planned so that one zone can carry all traffic if the other fails (often called N+1 capacity).
- During patching, extra servers are added first (surge) so capacity never drops below what customers need.

Why it matters for zero downtime: Having more than one copy of everything means one copy can be updated or fail while the others keep serving customers.

11. Zone B: server groups in the second data center

What it is: The same layout as Zone A, in a different, independent data center.

Simple analogy: The second kitchen, with its own power and staff.

What it does:

- Serves traffic at the same time as Zone A (active-active).
- Takes over all traffic automatically if Zone A has a problem.

Fine details:

- Updates are rolled out zone by zone, never to both at once, so a bad update can only ever affect a part of the system.
- Each zone has its own canary, so a zone-specific issue is spotted early.

Why it matters for zero downtime: Limits the 'blast radius' of any problem to a small, recoverable part of the system.

12. Database: Primary and Standby Replica

What it is: A managed database (for example AWS RDS Multi-AZ or Azure SQL with geo-replication) with a primary copy and a continuously synchronized standby in another zone.

Simple analogy: A main ledger and a carbon copy that is written at the same moment, kept in another building.

What it does:

- The primary handles reads and writes.
- The standby receives every change continuously (replication).
- If the primary fails or is being patched, the standby is promoted to primary automatically.

Fine details:

- Database patching is done standby-first: patch the standby, switch roles (a failover of a few seconds), then patch the old primary.
- Applications use retry logic and connection pooling so a brief switch is invisible to customers.
- Schema changes follow the 'expand then contract' pattern: add new columns first, switch code, remove old ones later. This keeps old and new app versions both working during a rollout.
- Backups and snapshots are taken in the pre-flight step before any risky change.

Why it matters for zero downtime: Data stays safe and available during patches and failures, and a rollback never leaves the data in a state the old version cannot read.

5.4 Safety loop components

13. Monitoring and Health Checks

What it is: Tools (for example CloudWatch, Prometheus and Grafana, Datadog) that measure how the system is behaving every few seconds.

Simple analogy: The patient monitor in a hospital that beeps when a vital sign goes wrong.

What it does:

- Collects the 'golden signals': error rate, latency (response time), traffic and saturation (CPU, memory, disk).
- Runs synthetic tests: robot users that log in, search or pay every minute, just like a real customer.
- Compares the canary group with the stable group side by side.

Fine details:

- Service Level Objectives (SLOs) define 'healthy', for example 99.9% of requests succeed and 95% finish in under 300 ms.
- Dashboards for each change show before, during and after so reviewers see the effect at a glance.

Why it matters for zero downtime: You cannot protect what you cannot see. Monitoring detects trouble before customers notice it.

14. Canary Analysis Gate

What it is: An automated decision step that compares the new version (canary) with the old one (stable) and returns pass or fail.

Simple analogy: The canary in a coal mine: a small early warning before everyone is exposed.

What it does:

- Waits a 'bake time' (for example 10 to 30 minutes) after each traffic step.
- Checks rules such as: canary error rate is not more than 0.5 percentage points above stable; p95 latency is no more than 10% higher; no crash loops; synthetic tests are passing.
- Returns pass (continue to the next step) or fail (roll back immediately).

Fine details:

- Can be built with tools such as Argo Rollouts, Flagger, Spinnaker/Kayenta or custom scripts that query the monitoring system.
- Decisions are automatic so no one has to stare at dashboards, and a human can also press 'stop' at any time.

Why it matters for zero downtime: Turns 'I think it looks fine' into a measured, repeatable yes or no.

15. Rollback and Failover Engine

What it is: The automation that undoes a bad change or works around a failed part, without waiting for a human.

Simple analogy: An undo button plus an automatic spare tire.

What it does:

- **Rollback**: sets canary traffic weight to 0%, points the server group back to the previous image ID, and re-applies the last known-good Terraform commit if needed.
- **Failover**: replaces unhealthy servers, moves traffic away from a failed zone, or promotes the database standby.
- Confirms the rollback worked by checking health again.

Fine details:

- Rollback is tested regularly (including game days or chaos drills) so it is trusted when it is needed.
- Typical time to undo a bad canary: a few minutes, while only a small share of traffic was ever exposed.

Why it matters for zero downtime: Mistakes can still happen, but their impact is small and short-lived, so customers do not experience downtime.

16. Alerts and Audit Log

What it is: Notifications (for example Slack or Teams messages, PagerDuty pages) and a permanent record of every action taken.

Simple analogy: The aircraft's black box plus the cockpit alarm.

What it does:

- Sends start, progress, success and failure messages to the team channel.
- Pages the on-call engineer if a rollback or failover happens.
- Writes all logs, plans, metrics snapshots and timings back to the change ticket as evidence.

Fine details:

- The evidence is what makes the 'zero customer-facing downtime' claim verifiable: each of the 300+ change records shows health data and the result.
- Post-incident reviews use this log to improve the runbooks.

Why it matters for zero downtime: Keeps people informed, proves the outcome and helps the system get better over time.

6. Inside the Terraform Code

Terraform is the foundation of the whole project. This section shows, in simple terms, how the code is organised and what a typical change looks like.

6.1 How the code is organised

Folder / file	What is inside	Why
modules/network	Virtual network, subnets in each zone, routing, firewalls	Same secure network everywhere
modules/load_balancer	Load balancer, listeners, target groups, health checks, draining delay	Traffic control for canary and rollback
modules/server_group	Launch template (image ID), auto scaling group, rolling update rules	Servers built from images, replaced safely
modules/database	Managed database, multi-zone standby, backups, maintenance window	Data safety and automatic failover
modules/dns	DNS records, health checks, failover routing	Site-level failover
modules/monitoring	Alarms, dashboards, SLO alerts	Health signals for the canary gate
envs/staging	Uses all modules with small sizes	Dress rehearsal environment
envs/production	Uses the same modules with production sizes	Identical design, so staging tests are meaningful
backend.tf	Where state is stored and locked	Shared, safe state
variables.tf / *.tfvars	Values that change: image ID, canary weight, instance counts	Most changes are just a new value here

6.2 Example: remote state with locking

This tells Terraform to keep its state in a shared, encrypted place and lock it during changes.

```
terraform {
  backend "s3" {
    bucket         = "company-terraform-state"
    key            = "production/web/terraform.tfstate"
    region         = "us-east-1"
    encrypt        = true
    dynamodb_table = "terraform-locks"   # one change at a time
  }
}
```

6.3 Example: weighted traffic for the canary

The load balancer sends traffic to two groups. Changing canary_weight from 5 to 25 to 50 to 100 is the ramp-up; setting it back to 0 is the rollback.

```
resource "aws_lb_listener_rule" "web" {
  listener_arn = aws_lb_listener.https.arn
  action {
    type = "forward"
    forward {
      target_group {
        arn    = aws_lb_target_group.stable.arn
        weight = 100 - var.canary_weight   # e.g. 95
      }
      target_group {
        arn    = aws_lb_target_group.canary.arn
        weight = var.canary_weight         # e.g. 5
      }
    }
  }
  condition {
    path_pattern { values = ["/*"] }
  }
}
```

6.4 Example: safe rolling replacement of servers

When the image ID changes, servers are replaced a few at a time, while at least 90% stay healthy and in service.

```
resource "aws_autoscaling_group" "web" {
  min_size          = 6
  max_size          = 12          # room to add new servers first
  health_check_type = "ELB"       # use load balancer health checks
  launch_template {
    id      = aws_launch_template.web.id   # contains var.image_id
    version = aws_launch_template.web.latest_version
  }
  instance_refresh {
    strategy = "Rolling"
    preferences {
      min_healthy_percentage = 90   # never drop below 90%
      instance_warmup        = 120  # seconds to warm up
      auto_rollback          = true # undo if refresh fails
    }
  }
  lifecycle { create_before_destroy = true }
}
```

6.5 Example: protecting critical resources

```
resource "aws_db_instance" "main" {
  multi_az                = true   # standby in another zone
  backup_retention_period = 14
  deletion_protection     = true
  lifecycle { prevent_destroy = true }  # Terraform refuses to delete it
}
```

> **Note on the examples**
>
> The snippets use AWS names because they are common and easy to read. The same design works on Azure, Google Cloud or on-premises with the equivalent Terraform resources. The exact values (5%, 90%, 120 seconds) are typical examples, not fixed rules.

7. The Four Automation Playbooks in Detail

The operational automation is made of four playbooks (scripted procedures). Each one is explained step by step.

7.1 Safe patching

Goal: apply operating system and software fixes without any customer noticing.

#	Step	Simple explanation
1	Pick a small batch	Choose a few servers, for example one per zone or 10% of the fleet.
2	Add spare capacity first (surge)	Start replacement servers before removing old ones, so total capacity never drops.
3	Drain	Tell the load balancer to stop sending new requests to the batch and let current requests finish.
4	Remove from service	Once drained, take the servers out of the load balancer.
5	Patch or replace	Usually replace with servers from the new patched image; otherwise install patches in place and reboot.
6	Health check	Server must pass several health checks in a row and app-level tests.
7	Return to service	Add back to the load balancer and watch metrics for a few minutes.
8	Repeat	Move to the next batch only if everything looks healthy; stop and roll back otherwise.

7.2 Canary rollouts

Goal: find problems with a new version while only a small share of customers can be affected.

Stage	Traffic to new version	Bake time (example)	Checks before moving on
Canary 1	5%, one zone	15 to 30 min	Errors, latency, crashes, synthetic tests vs stable
Canary 2	25%	15 min	Same checks plus resource usage

Stage	Traffic to new version	Bake time (example)	Checks before moving on
Canary 3	50%, both zones	15 min	Same checks plus business metrics (logins, orders)
Full	100%	30 to 60 min watch	Full verification; old version kept on standby

Example pass rules: canary error rate no more than 0.5 percentage points above stable; p95 latency no more than 10% higher; zero crash loops; all synthetic journeys passing. If any rule fails, the rollout stops and rolls back automatically.

7.3 Automatic failover

Goal: keep customers served when something breaks, whether during a change or at any other time.

Failure	Who notices	Automatic response	Typical time
One server crashes	Load balancer health check	Stop sending it traffic; auto scaling builds a replacement	Seconds to a few minutes
A whole zone has problems	Health checks across the zone	Load balancer sends all traffic to the healthy zone, which has spare capacity	Seconds
Primary database fails	Managed database service	Standby is promoted to primary; apps reconnect with retries	Typically under a minute or two
Whole site or region fails	DNS health checks	DNS points customers to the backup site	About 1 to 2 minutes (short TTL)

7.4 Rollbacks

Goal: return to the last known-good version quickly and reliably.

Rollback type	How it works	When used
Traffic rollback	Set canary weight to 0%. All traffic goes back to stable servers.	First and fastest response during a canary; seconds.
Image rollback	Point the server group back to the previous image ID and refresh.	If new servers already took more traffic.

Rollback type	How it works	When used
Code rollback	Revert the Git commit and re-apply Terraform from the previous version.	If infrastructure settings were changed.
Database safety	Expand-then-contract schema changes so the old app can still read data; snapshots as a last resort.	Always planned before data changes.

Why rollback is trusted

Rollbacks are automatic, tested in staging and in regular drills, and the previous version is kept warm during a hold period. A rollback is not a failure of the project: it is the system working as designed. A change that rolled back with no customer impact still counts as zero customer-facing downtime.

8. End-to-End Workflow

This is the life of a single change, from idea to closed ticket. The same flow was repeated for each of the 300+ updates. The diagram shows the order; the text that follows explains each step.

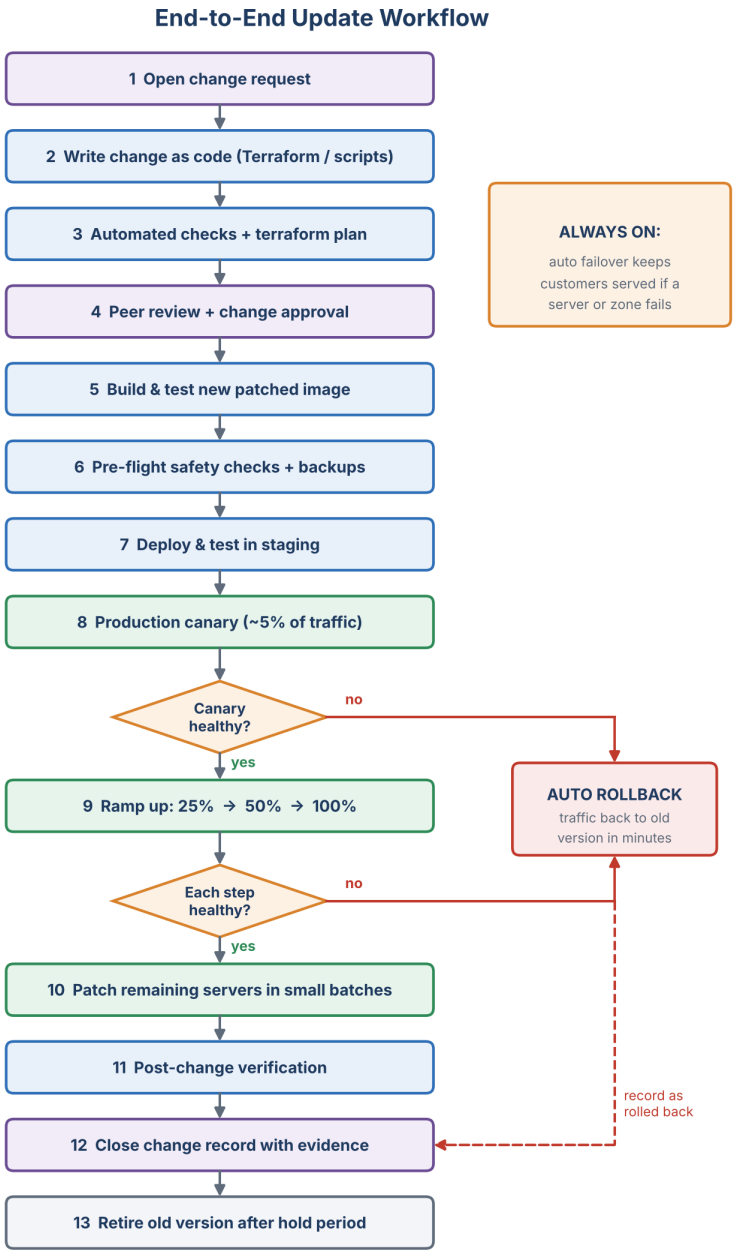


Figure 2. End-to-end update workflow with automatic rollback paths

8.1 Each step in detail

Step 1: Open a change request

Who or what does it: Engineer, in the change management system

What happens:

- Describe the change in plain words, for example 'Apply October OS security patches to the web tier'.
- State risk level, time window, affected services, test evidence and rollback plan.
- Link the ticket number in the Git branch and pull request.

In simple terms: Before doing anything, write down what you will do and how you will undo it.

Output: A change ticket in 'draft' or 'assess' state.

Step 2: Write the change as code

Who or what does it: Engineer, in Git

What happens:

- Edit Terraform variables or modules, for example set image_id to the new patched image.
- Update runbook settings if needed, for example batch size or bake time.
- Commit on a feature branch and open a pull request.

In simple terms: The change is a few lines of text, not a list of manual clicks.

Output: A pull request with a small, readable code change.

Step 3: Automated checks and terraform plan

Who or what does it: CI pipeline

What happens:

- Format, validate, lint and run security/policy scans.
- Run terraform plan against staging and production state.
- Post the plan summary (for example '0 to add, 2 to change, 0 to destroy') on the pull request and the ticket.

In simple terms: A robot proof-reads the change and shows exactly what it will do.

Output: A green build and a saved plan file.

Step 4: Peer review and change approval

Who or what does it: Reviewer(s), change approver or CAB

What happens:

- A teammate reviews the code and the plan.
- The change approver checks risk, timing and rollback plan.
- On approval, the pull request is merged and the production job is unlocked.

In simple terms: A second pair of eyes says 'yes, this is safe to try'.

Output: Merged code plus an approved change ticket.

Step 5: Build and test the new patched image

Who or what does it: Image pipeline (for example Packer) and registry

What happens:

- Start from the current base image, apply OS and package patches, install the app.
- Run automated tests and a vulnerability scan on the image.
- Publish it with a new version ID to the registry.

In simple terms: Build a brand-new, already-patched server template and test it before use.

Output: A tested image ID, for example app-2026-10-v14.

Step 6: Pre-flight safety checks and backups

Who or what does it: Orchestrator

What happens:

- Confirm the system is healthy right now (no open incidents, error rate normal).
- Confirm enough spare capacity exists to take servers out of service.
- Take database snapshots and save the current Terraform state version and image ID as the rollback point.
- Acquire the state lock so no other change can run.

In simple terms: Check the patient is stable and the spare tire is ready before starting surgery.

Output: A 'go' signal and a recorded rollback point.

Step 7: Deploy and test in staging

Who or what does it: Orchestrator, in the staging environment

What happens:

- terraform apply in staging, which is built from the same modules as production.
- Run smoke tests, integration tests and synthetic user journeys.
- Practise the rollback in staging if the change is new or higher risk.

In simple terms: Do a full dress rehearsal on a copy of production.

Output: Staging passes; results attached to the ticket.

Step 8: Production canary (about 5% of traffic)

Who or what does it: Orchestrator, load balancer, canary gate

What happens:

- Create a small canary group from the new image in one zone.
- Wait until the new servers pass load-balancer health checks.

- Shift about 5% of real traffic to the canary.
- Wait the bake time while monitoring compares canary against stable.

In simple terms: Let a few real customers use the new version while watching closely.

Output: Pass or fail from the Canary Analysis Gate.

Step 9: Ramp up step by step (25%, then 50%, then 100%)

Who or what does it: Orchestrator and canary gate

What happens:

- Increase the canary weight in steps, waiting and checking health after each one.
- Extend to the second zone only after the first zone looks good.
- Any failed check at any step triggers automatic rollback.

In simple terms: Open the door slowly, checking at every step that nothing is wrong.

Output: New version carrying all traffic, still with the old version on standby.

Step 10: Patch remaining servers in small batches

Who or what does it: Orchestrator, auto scaling, load balancer

What happens:

- For any servers patched in place, or remaining old servers: work one small batch at a time (for example 10% or one server per zone).
- For each batch: add a replacement (surge), drain connections, remove from load balancer, patch or replace, health-check, add back.
- Never take capacity below the safe minimum (for example at least 90% healthy).

In simple terms: Fix the train carriages one at a time while the train keeps running.

Output: All servers on the new version, capacity never reduced.

Step 11: Post-change verification

Who or what does it: Orchestrator and monitoring

What happens:

- Run full smoke tests and synthetic journeys again.
- Compare dashboards before and after: errors, latency, CPU, business metrics such as successful logins or orders.
- Run terraform plan again: it should show 'no changes', proving production matches the code.

In simple terms: Double-check everything is working as well as or better than before.

Output: Verification evidence.

Step 12: Close the change record with evidence

Who or what does it: Pipeline (automatic) and engineer

What happens:

- Attach logs, plan, metric snapshots and timings to the ticket.
- Mark the result: successful, successful with issues, or rolled back.
- Record customer impact, which for these 300+ changes was none.

In simple terms: Write the result in the official log book so it can be proven later.

Output: A closed change ticket.

Step 13: Retire the old version after a hold period

Who or what does it: Orchestrator / Terraform

What happens:

- Keep the old image and, where useful, the old server group for a hold period (for example 24 to 72 hours) for instant rollback.
- After the hold, scale the old group to zero and clean up.
- Release the state lock.

In simple terms: Keep the old parts for a little while just in case, then tidy up.

Output: A clean environment ready for the next change.

8.2 What happens when something goes wrong

Situation	What the system does	Customer experience
CI check or policy scan fails	Pull request is blocked; nothing reaches production.	None
Image fails tests or scan	Image is not published; deployment never starts.	None
Pre-flight finds the system unhealthy	Change is paused until the system is healthy again.	None
Staging tests fail	Stop; fix before production.	None
Canary looks worse than stable	Canary weight set to 0%, bad servers removed, team alerted.	At most a tiny share of requests see a brief error, retried automatically
A server fails during batch patching	Batch stops; replacement built; remaining servers still serve.	None (spare capacity covers it)
terraform apply errors halfway	State lock held; pipeline stops; previous commit re-applied.	None (old resources still serving)

Situation	What the system does	Customer experience
Zone or database failure during change	Automatic failover; change paused.	None or a few seconds of retries

8.3 A real-life style example: monthly security patch

- **Monday 10:00:** Engineer opens a standard change: 'October OS security patches for web tier'.
- **10:15:** Image pipeline builds app-2026-10-v14 with the patches; tests and scan pass.
- **10:30:** Pull request changes one line: image_id = 'app-2026-10-v14'. CI passes; plan shows '2 to change, 0 to destroy'.
- **11:00:** Reviewer approves; change ticket approved as a pre-approved standard change.
- **11:10:** Staging updated and all tests pass.
- **11:30:** Production canary: 5% traffic in Zone A. After 20 minutes, errors and latency match stable. Pass.
- **12:00 to 12:45:** Ramp to 25%, 50% (both zones), then 100%, each step passing.
- **13:00:** Verification passes; terraform plan shows 'No changes'. Evidence attached; ticket closed as successful, no customer impact.
- **Wednesday:** Old servers removed after the hold period.

9. How 'Zero Customer-Facing Downtime' Was Measured

The claim is backed by records, not memory. Each update had a change ticket, and each ticket contains:

- The planned change, the approval and the exact Git commit that ran.
- Pipeline logs with time stamps for every step.
- Health evidence before, during and after: error rate, latency, synthetic test results, availability against SLO.
- The outcome field: successful, or rolled back, and whether there was any customer impact.
- A cross-check that no customer-impacting incident was linked to the change.

Across 300+ change records, none showed a customer-facing outage. 'Customer-facing' is an important word: internal components were taken out of service many times (that is how patching works), but customers were always served by healthy copies.

Metric	What it shows
Number of in-service changes	300+ updates completed while live
Customer-facing downtime	Zero minutes recorded in change records
Change success rate	Share of changes completed without needing rollback
Rollbacks with no impact	Changes safely undone by automation before customers noticed
Mean time to roll back	How fast a bad change is undone (minutes)
Manual steps per change	Drops toward zero as automation grows

10. Explaining It in an Interview

10.1 Thirty-second version

> **Say it like this**
> "We needed to patch and upgrade a 24x7 service without maintenance windows. I codified the infrastructure in Terraform modules with remote, locked state and built pipeline automation for four things: batch patching with connection draining, weighted canary rollouts with automated health gates, multi-zone and database failover, and automatic rollback. Every run was tied to a change ticket with evidence. Over 300 in-service updates, change records show zero customer-facing downtime."

10.2 Likely follow-up questions and short answers

Question	Short answer
How did you avoid dropping requests?	Connection draining, surge capacity first, and min healthy percentage kept at about 90%.
How did you decide a canary was healthy?	Automated comparison of canary vs stable on error rate, p95 latency, crashes and synthetic tests over a bake time.
How fast was rollback?	Traffic rollback in seconds by setting the canary weight to 0; full image rollback in minutes.
How did you handle the database?	Multi-zone standby, standby-first patching, retries in the app, and expand-then-contract schema changes.
How did you stop risky Terraform changes?	Plan in every PR, policy-as-code scans, prevent_destroy on critical resources, and approvals.
How did you prevent drift?	No manual changes in production plus scheduled terraform plan drift checks.
Why trust the 'zero downtime' number?	It comes from change-management records with attached monitoring evidence for every change.